

Recurrent Neural Network (RNN)

Motivation

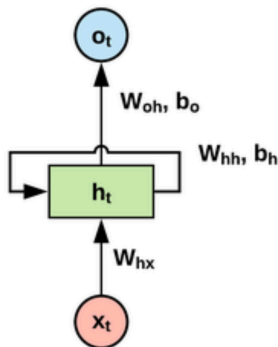
- Humans don't start their thinking from scratch every second.
- Vanilla neural network does not work with **sequential data**.



Opening or closing the door?

Introduction

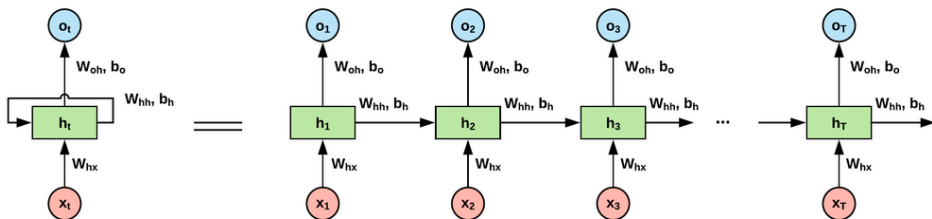
Recurrent neural network (RNN) can solve the issue: it has **memory** to **remember what it has seen** from sequential data.



RNN architecture

Introduction

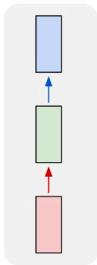
RNN is able to do: next word prediction, music composition, image captioning, speech recognition, time series anomaly detection, stock market prediction, etc.



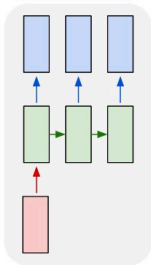
Unfold RNN

Introduction

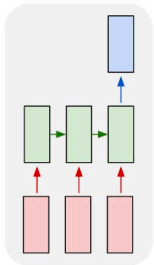
one to one



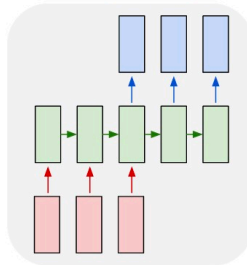
one to many



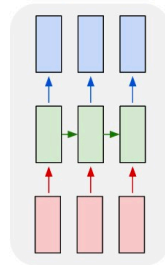
many to one



many to many



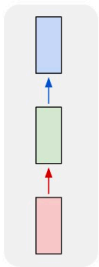
many to many



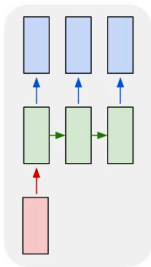
Different types of neural networks

Introduction

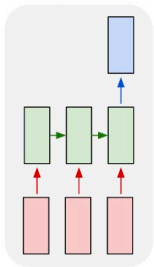
one to one



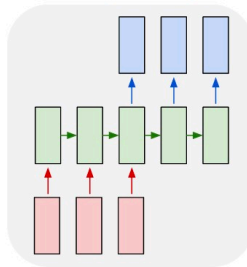
one to many



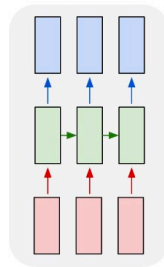
many to one



many to many



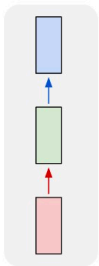
many to many



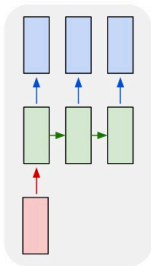
Vanilla Neural Networks

Introduction

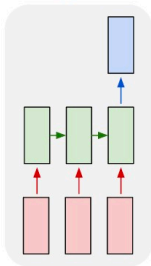
one to one



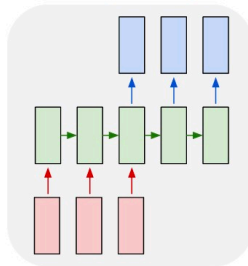
one to many



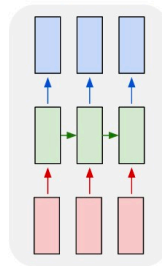
many to one



many to many



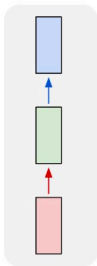
many to many



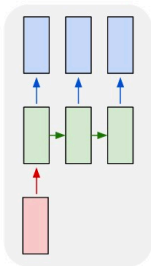
e.g., Image Captioning (image $\rightarrow$ sequence of words)

Introduction

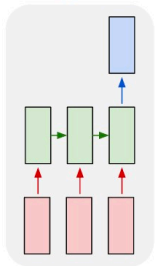
one to one



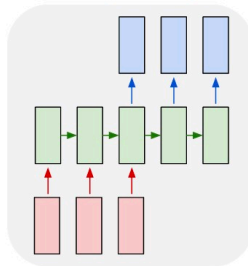
one to many



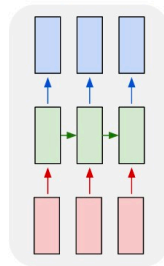
many to one



many to many



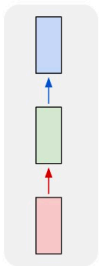
many to many



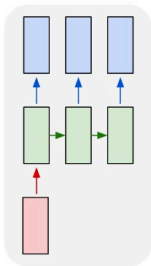
e.g., Sentiment Classification (sequence of words → sentiment)

Introduction

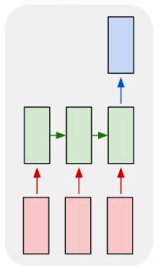
one to one



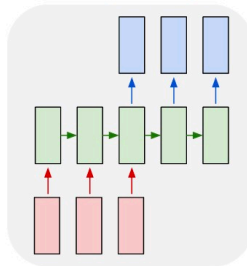
one to many



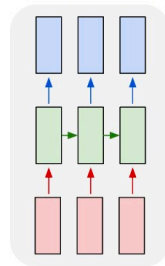
many to one



many to many



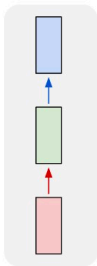
many to many



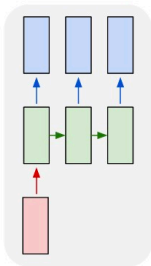
e.g., Machine Translation (seq of words $\rightarrow$ seq of words)

Introduction

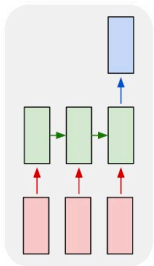
one to one



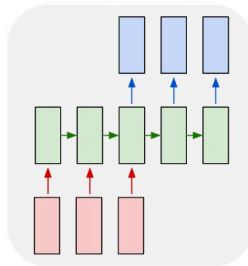
one to many



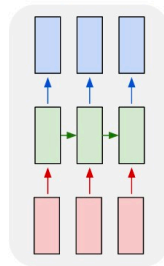
many to one



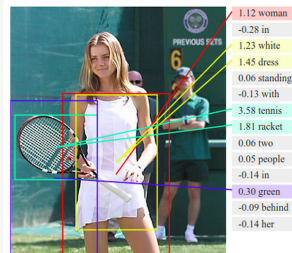
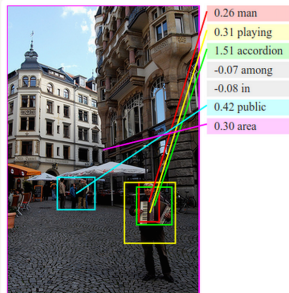
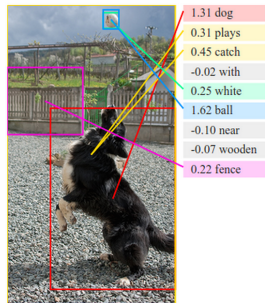
many to many



many to many



e.g., Video Classification on frame level



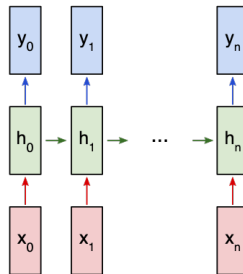
Deep Visual-Semantic Alignments for Generating Image Descriptions

RNN

Let's consider a "many to many" RNN with inputs $x_0, x_1, \dots, x_n$ that wants to produce outputs $y_0, y_1, \dots, y_n$. These x_i and y_i are **vectors** and can have arbitrary dimensions.

RNNs work by iteratively updating a hidden state h , which is a vector that can also have arbitrary dimension. At any given step t ,

1. The next hidden state h_t is calculated using the previous hidden state h_{t-1} and the next input x_t .
2. The next output y_t is calculated using h_t .



RNN

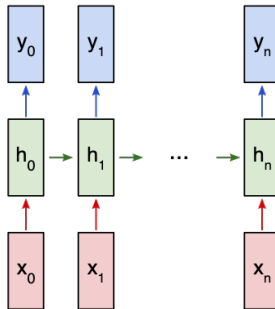
Here's what makes a RNN *recurrent*: **it uses the same weights for each step.**

More specifically, a typical vanilla RNN uses only 3 sets of weights to perform its calculations:

- W_{xh} , used for all $x_t \rightarrow h_t$ links.
- W_{hh} , used for all $h_{t-1} \rightarrow h_t$ links.
- W_{hy} , used for all $h_t \rightarrow y_t$ links.

We'll also use two biases for our RNN:

- b_h , added when calculating h_t .
- b_y , added when calculating y_t .



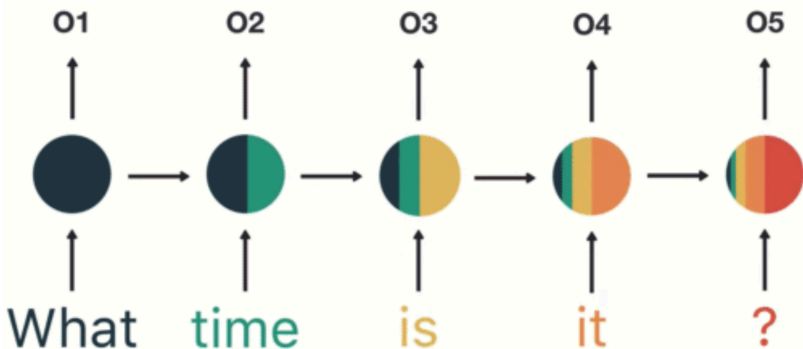
$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

Problem with RNN

Analyze the sentence “What time is it ?”

Asking for time



Problem with RNN

Analyze the sentence “What time is it ?”



Final hidden state of the RNN

The prediction is made mostly by “is it ?” → RNN is known as **short-term memory**